Minimizer - Assignment

Starting with the examples on github:

- Fitting-Example.ipynb
- ML scripts / notebooks

Develop:

1. A fitter which uses the χ^2 goodness-of-fit metric.
2. A “robust” minimizer
3. A gradient descent fitter

For all these, you will be tested with some input functions and evaluated in part on the code structure / readability and in part on the ability of your fitter/minimizer to reach the correct answer.

Fitter

The metric to test goodness of fit and converge should be the chisquare function:

$$\chi^2 = \sum_i \frac{(O_i - C_i)^2}{\sigma_i^2}$$

With O the data, C the prediction from the fit, and sigma the error on the data.

The fitter should be saved in a .py function with comments and a short description at the start.

The fitter needs to have the following syntax:

```
def fitter(f):
    …
    return r0,r1,chisq,ndof
```

With f a function with two parameters f(x,p0,p1), r0 and r1 the fit results for the two parameters, chisq and ndof the chisquare for the best-fit and the number of degrees of freedom in the probleb

Minimizer

Given a 1D function f(x) as input and a range of bounds [x_min, x_max], return the minimum of the function.

Your function should:

1. Have “memory”: keep track of the absolute minimum so far.
2. Randomly explore the neighborhood of your current location.
3. Try to “jump” away from the local area
    a. And probabilistically decide if to jump to that location based on the magnitude of the change in values. The worse your jump, the less likely you should choose to move there.

You will be evaluated based on (1) whether you reach the correct minimum, (2) how close to the absolute minimum you get, and (3) how quickly (number of iterations) you can get there.

To succeed in this test, you should try some dummy functions and figure out what parameters that allow you to reach the correct minimum.

Minimizer

Code format:

```
def minimize(f,tolerance=0.01):
    …
    return minval,niterations
```

The `tolerance` parameter is the level of convergence needed to satisfy the minimizer.

`minval` is the minimum that was found
`niterations` is the number of iterations it took to converge

You can get creative with your minimizer, but you need to code everything from scratch, without relying on external packages other than for random number generation.

Bonus: 2D minimizer

Expand your 1D minimizer to be able to find the minimum of a 2D function (on a 2D surface).

Can you use concepts from gradient descent to strategize and optimize how to reach the minimum of the function?